1	2	3	$\sum$

Freie Universität Berlin

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 05

Release: 12.11.2018; Due: 23.11.2018

Viktoriya Kraleva, Samuel Enderwitz

---

1 Specification and Implementation of an ADT (__/10)

Dies ist ein Abstrakten Datentyp, der als Warteschlange implementiert ist, aber anstatt die Elementen nur am Tail hinzugefügt werden zu können, kann man bei den Doppelendewarteschlangen sowohl am Ende, als auch am Beginn ein Element hinfügen oder entfernen.

$$\text{addBegin/insertFirst } x \in U \begin{cases} \text{Vorraussetzung:} & - \\ \text{Effekt:} & X' = x \mathbin{++} X \\ \text{Ergebnis:} & - \end{cases}$$

$$\text{addEnd/insertLast } x \in U \begin{cases} \text{Vorraussetzung:} & - \\ \text{Effekt:} & X' = X \mathbin{++} x \\ \text{Ergebnis:} & - \end{cases}$$

$$\text{getBegin/peekFirst} \begin{cases} \text{Vorraussetzung:} & X \neq \emptyset \\ \text{Effekt:} & X' = - \\ \text{Ergebnis:} & \text{erstes Element aus } X \end{cases}$$

$$\text{getEnd/peekLast} \begin{cases} \text{Vorraussetzung:} & X \neq \emptyset \\ \text{Effekt:} & X' = - \\ \text{Ergebnis:} & \text{letztes Element aus } X \end{cases}$$

$$\text{removeBegin/dequeueFirst} \begin{cases} \text{Vorraussetzung:} & X \neq \emptyset \\ \text{Effekt:} & X' = X \text{ ohne erstes Element aus } X \\ \text{Ergebnis:} & \text{erstes Element aus } X \end{cases}$$

$$\text{removeEnd/dequeueLast} \begin{cases} \text{Vorraussetzung:} & X \neq \emptyset \\ \text{Effekt:} & X' = X \text{ ohne letztes Element aus } X \\ \text{Ergebnis:} & \text{letztes Element aus } X \end{cases}$$

$$\text{size} \begin{cases} \text{Vorraussetzung:} & - \\ \text{Effekt:} & X' = - \\ \text{Ergebnis:} & |X| \end{cases}$$

$$\text{isEmpty} \begin{cases} \text{Vorraussetzung:} & - \\ \text{Effekt:} & X' = - \\ \text{Ergebnis:} & |X| = 0? \end{cases}$$

Sorry, dass es so viel Code geworden ist. Wir wollten die Aufgabe so vollständig wie möglich zu lösen.

Bei der ersten Implentierung mit dem Array haben wir eine maximale Größe. Unten haben wir noch eine alternative Implementierung mit einem DoubleDynamicArray.

```java
public interface DoubleQueueInterface<E>{
    E dequeueFirst() throws EmptyQueueException;
    E dequeueLast() throws EmptyQueueException;
    void insertFirst(E element) throws HeadIsTailException;
    void insertLast(E element) throws HeadIsTailException;
    E peekFirst() throws EmptyQueueException;
    E peekLast() throws EmptyQueueException;
    public int size();
    public boolean isEmpty();
}
```

Listing 1: DoubleQueueInterface

```java
public class EmptyQueueException extends Exception {
    public EmptyQueueException(String s){
        super(s);
    }
}
```

Listing 2: EmptyQueueException

```java
public class HeadIsTailException extends Exception {
    public HeadIsTailException(String s){
        super(s);
    }
}
```

Listing 3: HeadIsTailException

```java
import java.awt.*;

public class DoubleQueue<E> implements  DoubleQueueInterface {
    int head;
    int tail;
    E[] array;
    public DoubleQueue(int s) throws NegativeSize {
        if(s<0){
            throw new NegativeSize("Size must be positive!");
        }
        this.array = (E[]) new Object[s];
        this.head = 0;
        this.tail = 0;


    }
    @Override
    public E dequeueFirst() throws EmptyQueueException {
        if (!isEmpty()) {
            if(head==array.length) {
                head = 0;
            } else {
```

```java
                E element = (E) array[head];
                array[head]=null;
                head++;
            }
        } else {
            throw new EmptyQueueException("Cannot,queue is empty");

        }
        return null;
    }

    @Override
    public E dequeueLast() throws EmptyQueueException {
        if(!isEmpty()){
                E element =array[tail];
                array[tail]=null;
                tail--;
                if (tail<0){
                    tail=this.array.length-1;
                }

                return element;
            }
        else {
            throw new EmptyQueueException("Cannot,queue is empty");
        }
    }

    @Override
    public void insertLast(Object element) throws HeadIsTailException{
        if(isEmpty()) {
            tail=0;
        }else{
            tail++;
        }
        if(isfull()){
            throw new HeadlessException("The list is full");
        }
        if (this.array.length == tail) {
            this.tail = 0;
        }
        array[tail] = (E) element;
        }


    public boolean isfull(){
        return  ((head==0 && tail==array.length-1)||tail+1==head&&array[tail
    ]!=null);
    }


    @Override
    public void insertFirst(Object element) throws HeadIsTailException{
        array[head] = (E) element;
        head--;
        if(head<0){
            head=array.length-1;
```

```java
        }

        if(isfull()) {
            throw new HeadlessException("Cannot,the queue is full");
        }
        }

    @Override
    public E peekFirst() {
        return array[head];
    }

    @Override
    public E peekLast() {
        return array[tail];
    }

    @Override
    public int size() {
        return array.length;
    }

    @Override
    public boolean isEmpty() {
        return (head==tail&&array[head]==null);
    }


}
```

Listing 4: DoubleQueue

```java
import org.junit.Assert;
import org.junit.Rule;
import org.junit.Test;
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;
import static org.junit.jupiter.api.Assertions.assertThrows;
import org.junit.rules.ExpectedException;

import static org.hamcrest.CoreMatchers.containsString;

import java.awt.*;
import java.lang.reflect.Array;
import java.lang.reflect.Executable;
import java.util.Arrays;

public class TestDoubleQueue {
    @Test
    public void testDoubleQueue1() throws NegativeSize {
       try {
           DoubleQueue queue = new DoubleQueue<String>(3);
           Assert.assertFalse("should be empty", queue.isEmpty() == false);
       } catch (NegativeSize n){
           throw new NegativeSize("Size is negative");
       }
```

```java
    }
    @Test
    public void testDoubleQueue2() throws NegativeSize {
        try {
            DoubleQueue queue = new DoubleQueue<String>(3);
            queue.insertLast(new String("LOVE"));
            Assert.assertEquals("LOVE", queue.array[0]);
            } catch (NegativeSize n){
            throw new NegativeSize("Size is negative");
        } catch (HeadIsTailException e) {
            e.printStackTrace();
        }
    }
    @Test
    public void testDoubleQueue3() throws NegativeSize{
        try {
            DoubleQueue queue = new DoubleQueue<String>(3);
            queue.insertFirst(new String("MIMI"));
            queue.insertLast(new String("KOKO"));
            Assert.assertEquals("MIMI",queue.array[0]);
            Assert.assertEquals("KOKO",queue.array[1]);
        } catch (NegativeSize n){
            throw new NegativeSize("Size is negative");
        } catch (HeadIsTailException e) {
            e.printStackTrace();
        }
    }

    @Test(expected = HeadlessException.class)
    public void testDoubleQueue4() throws NegativeSize{
        try {
            //TODO FIX 4 and 5
            DoubleQueue queue = new DoubleQueue<String>(3);
            queue.insertLast(new String("BEBI"));
            queue.insertLast(new String("VICKY"));
            queue.insertLast(new String("ROBI"));
            Assert.assertFalse("Should not be empty",queue.isEmpty());
            Assert.assertEquals(3,queue.size());
            queue.insertLast(new String("Kotence"));

        } catch (NegativeSize n){
            throw new NegativeSize("Size is negative");
        } catch (HeadIsTailException e) {
            e.printStackTrace();
        }
    }
    @Test
    public void testDoubleQueue5() throws NegativeSize{
        try {
            DoubleQueue queue = new DoubleQueue<String>(3);
            queue.insertLast(new String("BEBI"));
            queue.insertLast(new String("BEBI1"));
            Assert.assertEquals("BEBI1",queue.array[1]);
            Assert.assertEquals("BEBI",queue.array[0]);
            queue.dequeueFirst();
            Assert.assertNull("Should be null",queue.array[0]);
            queue.dequeueLast();
```

```
            System.out.println(Arrays.toString(queue.array));
            Assert.assertNull("should be null",queue.array[1]);
            Assert.assertFalse("Should be true",queue.isEmpty());
        } catch (NegativeSize n){
            throw new NegativeSize("Size is negative");
        } catch (EmptyQueueException e) {
            e.printStackTrace();
        } catch (HeadIsTailException e) {
            e.printStackTrace();
        }
    }

}
```

Listing 5: TestDoubleQueue

```
public class ListNode<E> {
    E element;
    ListNode<E> next;
    ListNode<E> previous;

    public ListNode(Object element){
        this.element=(E) element;
    }

    public void setNext(ListNode<E> next) {
        this.next = next;
    }
    public void setPrevious(ListNode<E> previous){
        this.previous=previous;
    }
}
```

Listing 6: ListNode

```
public class DoubleLinkedList<E> implements DoubleQueueInterface<E> {
    ListNode<E> head;
    ListNode<E> tail;
    ListNode<E> current;
    int size;
    public DoubleLinkedList(){
        size = 0;
        this.head=this.tail=this.current=null;
    }
    @Override
    public E dequeueFirst() throws EmptyQueueException,NullPointerException {
        if(isEmpty()){
            throw new EmptyQueueException("List is empty,cannot delete");
        }else{
            E element = head.element;
            head=head.next;
            head.previous=null;
            size--;
            return element;
        }
    }

```

```java
    @Override
    public E dequeueLast() throws EmptyQueueException,NullPointerException {
        if(isEmpty()){
            throw new EmptyQueueException("List is empty,cannot delete");
        }else{
            ListNode<E> temp = tail;
            E element = tail.element;
            tail.previous.next=null;
            tail=temp.previous;
            size--;
            return element;
        }
    }

    @Override
    public void insertFirst(E element) throws HeadIsTailException,
   NullPointerException {
        ListNode<E> newest = new ListNode(element);
        if(isEmpty()){
            this.head=this.tail=new ListNode<E>(element);
        }else{
            ListNode<E> temp = head;
            head=head.previous=newest;
            newest.next=temp;
        }

        size++;
    }

    @Override
    public void insertLast(E element) throws HeadIsTailException,
   NullPointerException {
        if(isEmpty()){
            this.head=this.tail=new ListNode<E>(element);
            size++;
        }else {
            ListNode<E> newest = new ListNode(element);
            ListNode<E> temp = tail;
            tail=tail.next=newest;
            newest.previous=temp;
            size++;

            }
        }


    @Override
    public E peekFirst() {
        return head.element;
    }

    @Override
    public E peekLast() {
        return tail.element;
    }

    @Override
```

```java
    public int size() {
        return this.size;
    }

    @Override
    public boolean isEmpty() {
        return (head==tail&&tail==null);
    }

    public void printlist(){
        current=head;
        while (current!=null) {
            System.out.println(current.element);
            current=current.next;
        }
    }
}
```

Listing 7: DoubleLinkedList

```java
import org.junit.Assert;
import org.junit.Rule;
import org.junit.Test;
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

import org.junit.rules.ExpectedException;

import static org.hamcrest.CoreMatchers.containsString;
import static org.junit.jupiter.api.Assertions.*;

import java.awt.*;
import java.lang.reflect.Array;
import java.lang.reflect.Executable;
import java.util.Arrays;

public class TestDoubleLinkedList {
    @Test
    public void testDoubleLinkedList1() throws HeadIsTailException {
        DoubleLinkedList queue = new DoubleLinkedList();
        try {
            String first = new String("VIKI");
            String second = new String("ROBI");
            queue.insertFirst(first);
            Assert.assertEquals(first,queue.head.element);
            queue.insertLast(second);
            queue.printlist();
            Assert.assertEquals(second,queue.tail.element);
        } catch (HeadIsTailException e) {
            e.printStackTrace();
        }
    }
    @Test
    public void testDoubleLinkedList2(){
        DoubleLinkedList queue = new DoubleLinkedList();
        ListNode first = new ListNode("Viki");
```

```java
        ListNode second = new ListNode("Robi");
        try {
            queue.insertFirst(first);
            queue.insertLast(second);
            queue.dequeueFirst();
            Assert.assertNull(queue.head.previous);
            queue.insertFirst(first);
            queue.dequeueLast();
            Assert.assertEquals(first,queue.peekFirst());
        } catch (HeadIsTailException e) {
            e.printStackTrace();
        } catch (EmptyQueueException e) {
            e.printStackTrace();
        }
    }

    @Test
    public void testDoubleLinkedList3() throws HeadIsTailException {
        DoubleLinkedList queue = new DoubleLinkedList();
        ListNode first = new ListNode("Viki");
        ListNode second = new ListNode("Robi");
        queue.insertFirst(first);
        queue.insertLast(second);
        assertEquals(queue.peekFirst(),first);
        assertEquals(queue.peekLast(),second);
        assertEquals(queue.size,2);
    }
}
```

Listing 8: TestDoubleLinkedList

```java
import com.sun.xml.internal.ws.policy.privateutil.PolicyUtils;
import org.junit.Assert;
import org.junit.runner.*;
import org.junit.runner.notification.Failure;

import java.io.FileWriter;
import java.io.IOException;

import static jdk.nashorn.internal.objects.Global.print;


public class TestRunner {
    public static FileWriter fileWriter;

    static{
        try{
            fileWriter = new FileWriter("testResults.txt",true);
        } catch (IOException exc){
            exc.printStackTrace();
        }
    }

    public static void main(String[] args){
        print("Test stack");
        testDoubleQueue();
        testDoubleLinkedList();
```

```java
            try{
                fileWriter.close();
        } catch (IOException e){
            e.printStackTrace();
        }
    }

    private static void testDoubleLinkedList(){
        Result result = JUnitCore.runClasses(TestDoubleLinkedList.class);
        for(Failure failure : result.getFailures()){
            print(failure.toString());
        }
        int amount = result.getRunCount();
        int failed = result.getFailureCount();
        int ignored = result.getIgnoreCount();
        int succeded = amount - failed - ignored;
        print("Runned " + amount + " tests");
        print(succeded + " have succeded " + failed + " have failed "+ ignored + " were ignored ");
    }

    private static void testDoubleQueue(){
        Result result = JUnitCore.runClasses(TestDoubleQueue.class);
        for(Failure failure : result.getFailures()){
            print(failure.toString());
        }
        int amount = result.getRunCount();
        int failed = result.getFailureCount();
        int ignored = result.getIgnoreCount();
        int succeded = amount - failed - ignored;
        print("Runned " + amount + " tests");
        print(succeded + " have succeded " + failed + " have failed "+ ignored + " were ignored ");
    }

    private static void print(String message){
        System.out.println(message);
        try{
            fileWriter.write(message + '\n');
        } catch (IOException e){
            e.printStackTrace();
        }
    }
}
```

Listing 9: TestRunner

```java
public class NegativeSize extends Exception {
    public NegativeSize(String s){
        super(s);
    }
}
```

Listing 10: NegativeSize

Jetzt noch die alternative Implementierung mit einem in beide Richtingen dynamischer Array

```java
package de.smuu.add.e05;

public interface DoubleEndedQueue<E> {

    void addBegin(E element);
    void addEnd(E element);
    E getBegin() throws QueueEmptyException;
    E getEnd() throws QueueEmptyException;
    E removeBegin() throws QueueEmptyException;
    E removeEnd() throws QueueEmptyException;
    int size();
    boolean isEmpty();

}
```

Listing 11: DoubleEndedQueue

```java
package de.smuu.add.e05;

public class ArrayEmptyException extends RuntimeException {

    public ArrayEmptyException(String message) {
        super(message);
    }
}
```

Listing 12: ArrayEmptyException

```java
package de.smuu.add.e05;

import java.util.Arrays;

public class DoubleDynamicArray<E> {

    private E[] array;
    private int first, last;

    public DoubleDynamicArray() {
        this(5);
    }

    public DoubleDynamicArray(int size) {
        array = (E[]) new Object[size];
        first = size/2 -1;
        last = size/2;
    }

    public void addBegin(Object element) {
        if (first == -1) {
            resize();
        }
        array[first] = (E) element;
        first--;
    }

    public void addEnd(Object element) {
        if (last == array.length) {
```

```java
            resize();
        }
        array[last] = (E) element;
        last++;
    }

    public Object getBegin() throws ArrayEmptyException {
        if (isEmpty()) {
            throw new ArrayEmptyException("There are no elements in the DoubleEndedQueue");
        }
        return array[first+1];
    }

    public Object getEnd() throws ArrayEmptyException {
        if (isEmpty()) {
            throw new ArrayEmptyException("There are no elements in the DoubleEndedQueue");
        }
        return array[last-1];
    }

    public Object removeBegin() throws ArrayEmptyException {
        if (isEmpty()) {
            throw new ArrayEmptyException("There are no elements in the DoubleEndedQueue");
        }
        Object element = getBegin();
        first++;
        return element;
    }

    public Object removeEnd() throws ArrayEmptyException {
        if (isEmpty()) {
            throw new ArrayEmptyException("There are no elements in the DoubleEndedQueue");
        }
        Object element = getEnd();
        last--;
        return element;
    }

    public int size() {
        return last-first-1;
    }

    public boolean isEmpty() {
        return size()==0;
    }

    /**
     * It only called if really need to resize; so no check
     */
    private void resize() {
        System.out.println(this);
        // +1 because we want to add more and have to resize before
        int newSize = (size()+1)*2;
```

```java
        //newSize = 7;
        E[] newArray = (E[]) new Object[newSize];


        int newFirst = (int) Math.floor((newSize - size()) / 2.0) - 1;
        int newLast = newSize - (int) Math.ceil((newSize - size()) / 2.0);

        /*System.out.println("s: " + size());
        System.out.println("s: " + newSize);
        System.out.println("f: " + newFirst);
        System.out.println("l: " + newLast);*/

        int j = first+1;
        for (int i = newFirst+1; i <= newLast-1; i++) {
            newArray[i] = array[j];
            j++;
        }

        array = newArray;
        first = newFirst;
        last = newLast;

        System.out.println(this);
    }

    @Override
    public String toString() {
        return Arrays.toString(array);
    }
}
```

Listing 13: DoubleDynamicArray

```java
package de.smuu.add.e05;

public class QueueEmptyException extends RuntimeException {

    public QueueEmptyException(String message) {
        super(message);
    }
}
```

Listing 14: QueueEmptyException

```java
package de.smuu.add.e05;

public class DoubleArrayQueue<E> implements DoubleEndedQueue {

    private DoubleDynamicArray dNA;

    public DoubleArrayQueue() {
        this(5);
    }

    public DoubleArrayQueue(int size) {
        dNA = new DoubleDynamicArray(size);
    }
```

```java

    @Override
    public void addBegin(Object element) {
        dNA.addBegin(element);
    }

    @Override
    public void addEnd(Object element) {
       dNA.addEnd(element);
    }

    @Override
    public Object getBegin() throws QueueEmptyException {
        if (isEmpty()) {
            throw new QueueEmptyException("There are no elements in the DoubleEndedQueue");
        }
        return dNA.getBegin();
    }

    @Override
    public Object getEnd() throws QueueEmptyException {
        if (isEmpty()) {
            throw new QueueEmptyException("There are no elements in the DoubleEndedQueue");
        }
        return dNA.getEnd();
    }

    @Override
    public Object removeBegin() throws QueueEmptyException {
        if (isEmpty()) {
            throw new QueueEmptyException("There are no elements in the DoubleEndedQueue");
        }
        return dNA.removeBegin();
    }

    @Override
    public Object removeEnd() throws QueueEmptyException {
        if (isEmpty()) {
            throw new QueueEmptyException("There are no elements in the DoubleEndedQueue");
        }
        return dNA.removeEnd();
    }

    @Override
    public int size() {
        return dNA.size();
    }

    @Override
    public boolean isEmpty() {
        return dNA.isEmpty();
    }

```

```
    @Override
    public String toString() {
        return dNA.toString();
    }
}
```

Listing 15: DoubleArrayQueue

2 Lower Bounds for Priority Queues (__/5)

(a) *Show how to use a priority queue in order to sort a sequence of n pairwise comparable elements.*

Wir werden hier verbal beschreiben, was man in diesem Fall tut soll. Aus dem Grund,dass wir so eine Prioritätswarteschlange schon in TI2 mit Prozessen und verschiedene Strategien zum Auswahl der Reihenfolge, in der die Elemente bedient werden müssen,schon implementiert haben, können wir auch dazu noch eine C Datei mit verschiedene Ersetzungsstrategien hinzufügen (FCFS, RR, SPT..). Aus dem Grund,dass auch die Prozessen paarweise nicht vergleichbar sind,weil z.B manche auf dem Drucker ausgeführt werden müssen und andere auf dem Prozessor, braucht man einen anderen Merkmal,sodass man leichter die Warteschlange verwalten kann. Wir werden alle Elemente in einem Struktur (bzw. in Java Objekt) auffassen und zu jedem Element eine Priorität zuordnen im Anhang dazu, ob das Element so schnell wie möglich bedient werden muss oder nicht.
Mit der Zuordnung der Prioritäten,die als Integers zugewiesen werden (kleinste Wert = höchste Priorität) kann man jetzt alle Elemente anhand ihrer Prioritäten vergleichen. Dies werden wir mit dem Heap-Sort Algorithmus machen und aus dem Grund,dass wir zu n Elementen genau n Prioritäten zugeordnet haben (paarweise verschieden),können wir dies genau in $O(n * \log n)$ zeit machen.
Nun werden wir von unserem Objekt (Struktur) die queue.elementk.prioritat werten vergleichen.
Auf dieser Weise kann man eine paarweise verschiedene Elemente mit Hilfe Prioritätswarteschlangen vergleichen.Natürlich werden die nur nach ihrer Prioritätswert geordnet sein.

(b) *In* AlP2*, you learned that every comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons. Conclude that in every comparison-based implementation of a priority queue, there is at least one operation with $\Omega(\log n)$ amortized running time.*

Erstmal wissen wir, dass sowohl $A_{1 \leq i \leq m} = \frac{T(m)}{m}$. Jetzt betrachten wir die Quantorisierte Aussage der Omega Notation:
$\exists c \wedge n_0 : 0 \leq c * g(n) \leq f(n) \forall n > n_0$
Das heißt,dass $T(n) \geq \Omega(n * \log n)$
$A_{1 \leq i \leq n} = T(n)/n \geq \frac{\Omega(n * \log(n))}{n} = \Omega(\log n)$
Die Aussage folgt aus der beiden Definitionen,wenn man dies mit werten ersetzt. Also $T(n)_{amortisiert} \in \Omega(\log n)$ amortisiert.

3 Binary Heaps (__/15)

(a) *What is the minimum number of elements a binary heap with height h can have? What about the maximum number? Conclude that a binary heap with n elements must have height $\Theta(\log n)$.*
Attention: *We count the levels from* 0.
Wir wissen,dass ein Binärheap nichts anderes als ein Binärbaum ist.Deswegen werden wir erstmal die maximale Anzahl der Elemente bestimmen und dies ist genau der Fall, wenn der Binärbaum vollständig ist.
Der Formel haben wir in ALP 1 schon gelernt und damit ist die Maximale Anzahal : $E(h) \leq 2^{h+1} - 1$,wo E(h) die Anzahl der Elementen bezeichnet.
Dies muss mit Induktion bewiesen werden :
I.A. $n = 1, h = 0 \rightarrow 1 \leq 2^1 - 1 = 1$ -wahr.
I.V. Wahr für (h-1) $n \leq 2^h - 1$
I.S. $h - 1 \rightarrow h$
Hier müssen wir nun merken, dass wir genau $n + 1$ neue Blätter im schlimmsten Fall haben werden(beim vollständigen Binärenbaum).
Damit muss :
$n + n + 1 \leq 2^{h+1} - 1$ $2(n + 1) \leq 2 * 2^h$
Damit teilen wir durch 2:
$n + 1 \leq 2^h$
$n \leq 2^h - 1$ Hier ist die Aussage laut IV wahr.
Beim Höhe h könnte man entweder auf der linken oder auf der rechten Seite, in dem Blätter des Baums nur einen Blatt haben. Man könnte auch sowar überlegen, dass der Baum nur ein Pfad auf der linken Seite hat und alle Elemente linken (bzw. rechten) Kindern sind, aber damit wäre der Struktur kein Heap, weil der Wurzel nicht richtig ausgewählt wurde.
Mann kann die Höhe h erreichen, wenn man nur einen Blatt bei der Ebene h hat. Damit kann man mit Hilfe der Folgenden Formel die Anzahl der Elemente für die Höhe $h - 1$ berechnen und dazu 1 addieren und auf dieser Weise wird man der minimalen Anzahl an Elementen kriegen. Damit ist $min(E(h)) = max(E(h - 1)) + 1 = 2^{h-1} - 1 + 1 = 2^{h-1}$
Damit ist die Anzahl der Elementen $2^{h-1} \leq E(h) \leq 2^{h+1} - 1$
Die dritte Schlussfolgerung kriegen wir wie folgt :
$2^{h-1} \leq n \leq 2^{h+1} - 1$ (logarithmieren mit Basis 2 auf beiden Seiten)
$\log_2 2^{h-1} \leq \log_2 (n + 1) \leq \log_2 2^h$
$h - 1 \leq \log_2 (n + 1) \leq h + 1$
$h \leq \log_2 (n + 1) \leq h$
Damit ist $h = \log_2 (n + 1)$ und damit in $\log_2 (n + 1) \in \Theta(\log_2 n)$ - offensichtlich warum.

(b) *Where in a binary heap can the second smallest element be located? What about the third smallest element? Provide a general formula for the levels that may contain the k-smallest element in a binary heap with n elements. (You may assume that all levels are filled completely).*
Es hängt von dem Art des Heaps ab. Wenn es Minheap ist, ist für den zweiten sehr

leicht.Er befindet sich entweder in dem linken oder in dem rechten Kindknoten des Wurzeln.Der dritte kleinste Element muss in diesem Fall entweder in dem rechten Kind des Wurzels sein, oder in dem linken Kind des linken Kinds des Wurzeln oder in dem rechten Kind des linken Kinds des Wurzeln .
Aus dem Grund,dass man den 4.ten und 5. ten kleinsten Element nicht eindeutig bestimmen kann, werde ich eine Quantorenformel hier aufbauen und zwar muss das k kleinste Element $\in \lceil \log_2 (k+1) \rceil \vee \lceil \log_2 (k+1) \rceil - 1$ Ebene bzw. Höhe sein. (Die Höheformel haben wir schon oben bewiesen.)

(c) *Illustrate the bottom-up heap construction for* `[7, 6, 5, 3, 1, 4, 2, 8]`.
Hier veranschauligen wir unser Bottom-down Konstruktion $right(7) = 16$
$left(7) = 15$
$right(6) = 14 left(6) = 13$
$right(5) = 12$
$left(5) = 11$
$right(4) = 10$
$left(4) = 9$
Dann müssen wir den Aufruf von bubble-down bei $i = 3$ betrachten. Nur $left(i) < size$,aber $array[left(i)] > array[i]$ deswegen hört die Schleife auf.
Dann rufen wir $bubble - down(2)$ und dann ist $array[2] = 5$
die Bediengung ist im ersten Schleifendurchlauf erfüllt und nach dem Schleifendurchlauf sieht der Array wie folgt aus :
$[7, 6, 2, 3, 1, 4, 5, 8]$
Hier wird der zweiten Durchlauf abgebrochen, weil $i = 6 \wedge right(i) \wedge left(i) > size$
Dann rufen wir bubble-down mit $i = 1$ und $array[i] = 6$. Hier wird nach dem ersten Schleifendurchlauf die Array wie folgt aussehen :
$[7, 1, 2, 3, 6, 4, 5, 8]$
aber beim zweiten bricht die Schleife ab, weil $i = 4$ und $right(i) \wedge left(i) > size$
Dann ruden wir bubble-down mit $i = 0$ und $array[i] = 7$. Nach dem ersten Schleifedurchlauf sieht die Array wie folgt aus:
$[1, 7, 2, 3, 6, 4, 5, 8]$
Dann wird den zweiten Schleifendurchlauf mit $i = 1 \wedge array[i] = 7$ aufgerufen und am Ende sieht die Array wie folgt aus :
$[1, 3, 2, 7, 6, 4, 5, 8]$
Dann beim dritten Durchlauf hört die Schleife auf,weil $size < left(i)$,aber $array[left(i)] > array[i]$ und damit ist die Bediengung nicht erfüllt. Dazu noch ist $right(i) == size$ und dies erfüllt die Schleifebediengung auch nicht. Dann am Ende sieht die Array so aus :
$[1, 3, 2, 7, 6, 4, 5, 8]$
(Dies haben wir nur für Veranschauligung der nächsten Aufgabe hinzugefügt) und der Heap-Baum so:

Veranschauligung für die nächste Aufgabe Mann kann ja den Heap so aufbauen, indem man erstmal nur das erste Element als Heap betrachtet. Dann fügt man den zweiten und bildet ein Heap aus den beiden.Dann bei dem dritten macht man genau das gleiche und mit Hilfe der Bubble-Up Funktion das ganze nochmal in einem Heap umwandelt und sw. Wir veranschaulichen kurz die Idee dahinter mit den ersten drei

Elementen:
$[7]$
$[7, 6] \rightarrow [6, 7]$
$[6, 7, 5]$
$[6, 7, 5, 3] \rightarrow (hochbubblern)[6, 3, 5, 7] \rightarrow [3, 6, 5, 7]$ und sw.

(d) *Suppose we insert n elements successively into a binary heap that is initially empty. Show that this takes $\Omega(n \log n)$ time in the worst case.*
Attention*: Initially, an insertion takes much less than $\log n$ steps.*
Wir wissen schon,dass beim Bottom-Up Konstruktion wir linearen zeitlichen Aufwand brauchen $\in O(n)$ (schon in der Vorlesung bewiesen). Dann werden unser Worst-Case beim zweiter Möglichkeit,die in 3c) veranschauligt ist. Mann betrachtet am Anfang nur ein Element als Heap und erweitert den Heap mit jedem weiteren Element.
Nun betrachten wir wie viele Ebenen wir am Ende haben werden $h = \lceil \log_2 n + 1 \rceil$,aber wir wissen,dass wir genau $\lceil \log_2 n + 1 \rceil - 1$ Ebenen hochgehen dürfen,weil wir schon das letzte Element auf die unterste Ebene hinzufügen. Wir haben insgesamt n Elementen. Damit wäre den schlimmsten Fall wenn diese Verschiebung sich jedes Mal passiert und zwar bei jedem Hinzufügen von einem Element (also wenn die größten Elementen ganz vorne in der Array stehen).Damit werden wir im schlimmsten Fall genau bei $n-1$ Elementen diese $bubble-up$ Funktion mit Vertausch der Elementen haben.Dies führt auch zu rekursiven Aufrufen der Funktion, und damit auch $T_{bubbleup} = \log_2 n \in \Theta(\log_2 n)$.
Wir wissen schon,dass *bubbleup*'s running time $\Theta(h) = \Theta(log_2 n)$ ist,weil es genau $\lceil \log_2 n + 1 \rceil - 1$ rekursive Aufrufe am meisten geben kann.Damit können wir behaupten,dass den "worst case" running time für $buildheap = n * \log_2 n \in \Omega(n * log_2 n)$ sein wird und zwar wenn bei jedem Einfügen von einem Element, dies einen rekursiven Aufruf der *bubbleup* Funktion verursacht.